# Mastering I2C

Inter-Integrated Circuit Bus — A Complete Reference Guide
Signal Lines • START/STOP • Addressing • Frame Structure • Interview Q&A

Prepared for automotive embedded software engineering practice
(sensor/EEPROM/RTC/IMU interfacing, ECU peripheral communication)

# Table of Contents

# Part 1 — I2C Fundamentals

## 1.1 What is I2C and why it's used

I2C (Inter-Integrated Circuit, also written I²C, pronounced "I-squared-C" or "I-two-C") is a synchronous, half-duplex, multi-master, multi-slave serial bus developed by Philips (now NXP) in the 1980s. Unlike SPI (one dedicated wire pair per function, no addressing, master must actively select each slave with a separate CS pin) and unlike CAN (differential pair, message-based broadcast with arbitration on every frame), I2C uses just **two shared, open-drain wires** for the entire bus, and every device on it is distinguished by a unique **bus address** sent in-band rather than by a dedicated wire.

Why it's common in embedded/automotive designs:

- **Minimal pin count:** only 2 wires (SDA, SCL) no matter how many devices are on the bus — this is I2C's headline advantage over SPI, which needs an extra CS pin per device.
- **Built-in addressing and ACK:** every byte is acknowledged by the receiver, giving basic error/presence detection that SPI lacks entirely.
- **Multi-master capable:** multiple controllers can share the same bus with a defined arbitration scheme (unlike SPI, which has no standard way to support more than one master).
- **Ubiquitous peripheral support:** EEPROMs, RTCs, IMUs/accelerometers, temperature sensors, PMICs, and many other low/medium-speed peripherals default to I2C because of its low pin count and simple 2-wire wiring.

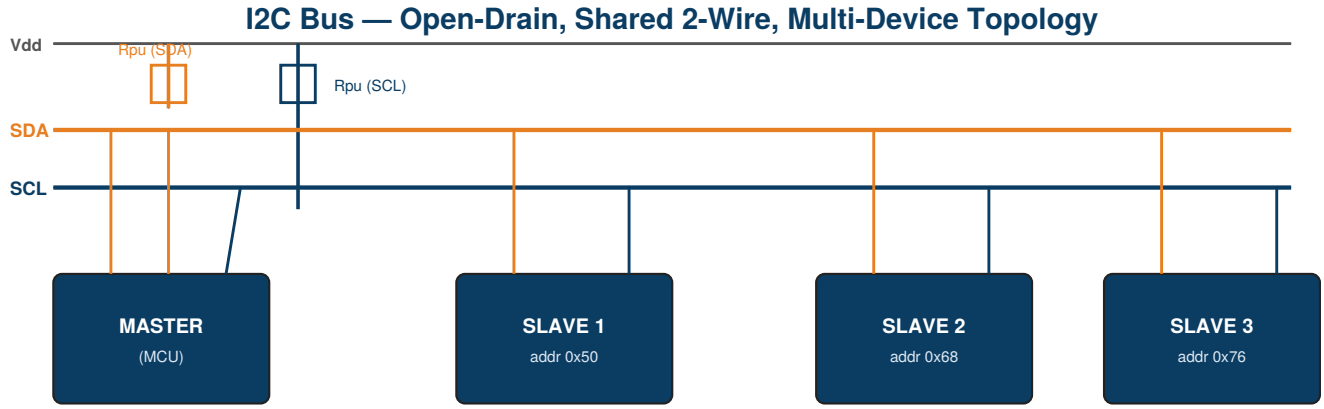
> Trade-off versus SPI: I2C is slower (typically 100 kHz-3.4 MHz vs SPI's tens of MHz) and half-duplex, because of the shared open-drain bus and per-byte addressing/ACK overhead. Choose SPI when raw throughput matters (flash memory, displays); choose I2C when pin count matters more than speed (many low-rate sensors on one bus).

## 1.2 The Two Signal Lines

| Signal | Full name | Driven by | Purpose |
|---|---|---|---|
| **SDA** | Serial Data | Master AND slave (shared, open-drain) | Carries all address and data bits, in both directions, at different points in the same transaction |
| **SCL** | Serial Clock | Master (normally); slave can hold it low to stretch | Clocks every bit; unlike SPI, a slave is also allowed to actively pull SCL low to pause the master (clock stretching — see Part 4) |

## 1.3 Open-Drain Bus & Pull-Up Resistors

Both SDA and SCL are **open-drain** (sometimes called open-collector) outputs: a device can only actively pull the line LOW, never actively drive it HIGH. External pull-up resistors (one per line, shared by the whole bus) are what pull the lines back HIGH when no device is pulling them low. This is a deliberate design choice — it's what allows multiple devices to share the same two wires without ever having two outputs directly fight each other (unlike SPI's push-pull MISO, which requires careful tri-stating to avoid contention).




Only 2 wires total (SDA + SCL), shared by ALL devices. Both lines are open-drain — pulled HIGH by resistors, actively pulled LOW by any device. Every device is distinguished purely by its unique bus address, not by a wire.

> Because it's a wired-AND bus (any device pulling low wins, since nothing can override a low with a high), this is also the electrical basis for both slave ACK/NACK signaling and multi-master arbitration — the same trick CAN uses for dominant/recessive arbitration, applied here to a single-ended two-wire bus instead of a differential pair.

## 1.4 Pull-Up Resistor Value — Why It Matters

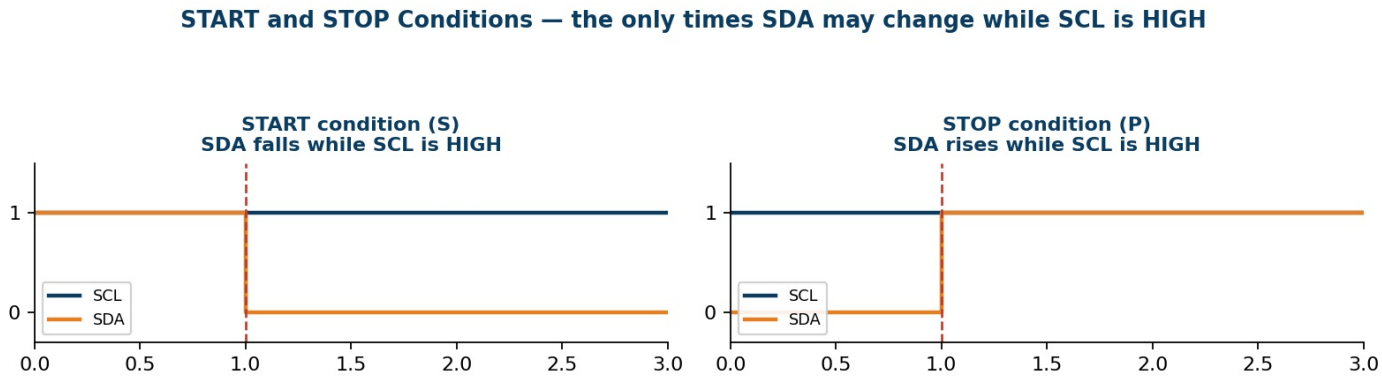
Too large a pull-up value means the line rises from LOW to HIGH too slowly (limited by R × bus-capacitance time constant), capping the maximum usable clock speed. Too small a value means excessive current draw whenever a device pulls the line low, and can exceed a device's rated sink current. Typical values are 2-10kΩ for standard/fast mode buses, chosen based on total bus capacitance and target speed — this is a real hardware design calculation ($t_{rise} \approx 0.8473 \times R \times C$ for a standard RC charging curve to the required logic threshold), not just "pick something safe."

# Part 2 — START, STOP, and Bus Idle/Busy Rules

## 2.1 The Golden Rule of I2C Timing

Every I2C transaction structure hinges on one rule: **SDA may only change state while SCL is LOW.** While SCL is HIGH, SDA must remain stable — a receiver is actively sampling it. The *only two exceptions* to this rule are the START and STOP conditions themselves, which are deliberately defined as SDA transitions *while SCL is HIGH* — this is precisely what makes them unambiguous, unmistakable framing markers that can never be confused with a normal data bit.

## 2.2 START and STOP Conditions



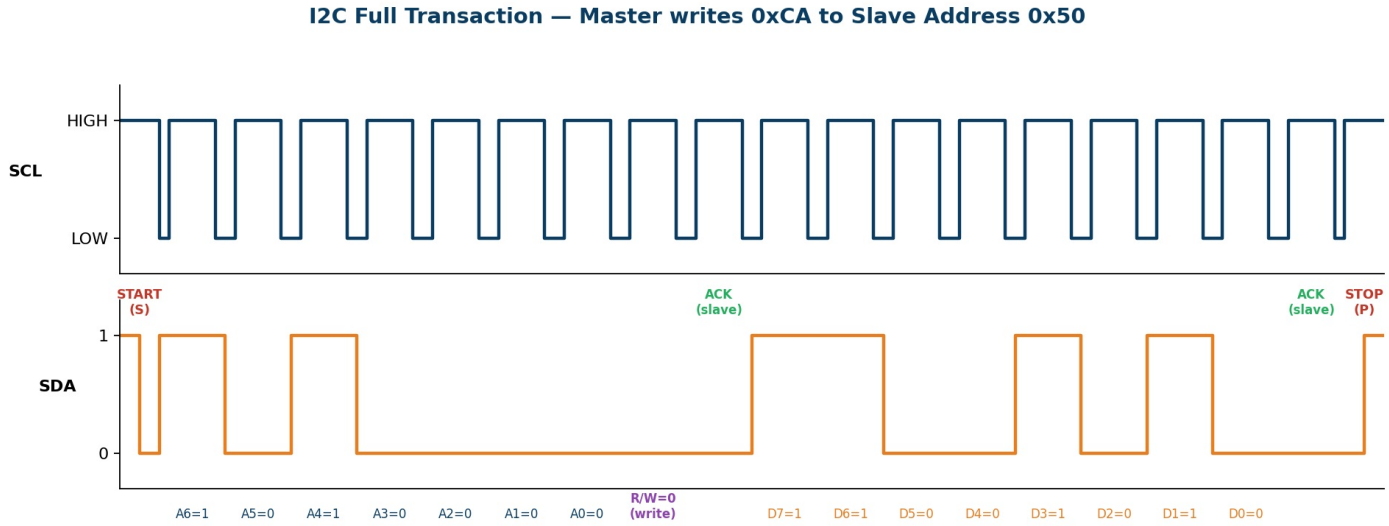

| Condition | Symbol | Definition | Meaning |
|---|---|---|---|
| **START** | S | SDA transitions HIGH→LOW while SCL is HIGH | Master signals the beginning of a new transaction; bus goes from "idle" to "busy" |
| **STOP** | P | SDA transitions LOW→HIGH while SCL is HIGH | Master signals the end of the transaction; bus returns to "idle," freeing it for any master to start a new transaction |
| **Repeated START** | Sr | A START condition issued without a preceding STOP | Lets the master immediately begin a new transaction (often switching from write to read, e.g. for a register-read sequence) without releasing the bus in between — prevents another master from grabbing the bus mid-sequence |

### Worked example — why Repeated START matters

A very common I2C pattern is "write the register address you want, then read its value" — e.g., reading register 0x0F from a sensor at address 0x50: (1) START, (2) address byte 0x50 with R/W=0 (write), ACK, (3) register address byte 0x0F, ACK, (4) **Repeated START** (not a STOP!), (5) address byte 0x50 with R/W=1 (read), ACK, (6) slave sends the register data byte, master sends NACK (or ACK if reading more bytes) to end the read, (7) STOP. Using a Repeated START instead of a full STOP+START here guarantees no other master can interleave a transaction between the "tell it which register" step and the "read it back" step — without this guarantee, another master could sneak in and change what register is selected before your read happens.

# Part 3 — Addressing and Frame Structure

## 3.1 Full Transaction — Bit by Bit




This shows a complete write transaction: master writes byte `0xCA` to the slave at 7-bit address `0x50`. Reading it field by field:

| # | Field | Width | Meaning |
|---|---|---|---|
| 1 | START (S) | — | SDA falls while SCL is high; announces a new transaction is beginning. |
| 2 | Slave Address | 7 bits | Sent MSB-first. Identifies exactly one device on the shared bus — the electrical equivalent of SPI's dedicated CS wire, but sent in-band instead. |
| 3 | R/W bit | 1 bit | 0 = master is about to WRITE to the slave. 1 = master wants to READ from the slave. This is the 8th bit sent right after the address, so the full first byte on the wire is "address+R/W." |
| 4 | ACK (from slave) | 1 bit | The addressed slave pulls SDA LOW during this 9th clock pulse to acknowledge "yes, I'm here and I recognize my address." If no device responds (wrong address, device not present/powered), the line stays HIGH — a NACK, and the master knows immediately that nothing answered. |
| 5 | Data byte | 8 bits | Sent MSB-first, direction determined by the R/W bit sent earlier (master drives it for a write, slave drives it for a read). |
| 6 | ACK (from receiver) | 1 bit | Whichever side is RECEIVING this byte acknowledges it — for a write, the slave ACKs; for a read, the master ACKs (or NACKs on the last byte it wants, telling the slave to stop sending). |
| 7 | STOP (P) | — | SDA rises while SCL is high; ends the transaction, freeing the bus. |

> Notice the pattern: every single byte on an I2C bus — whether it's the address byte or a data byte — is always followed by exactly one ACK/NACK bit. This is completely unlike SPI (no acknowledgment at all) and unlike CAN (one shared ACK per whole frame, not per byte).

## 3.2 7-bit vs 10-bit Addressing

| Addressing | Address space | How it's sent |
|---|---|---|
| **7-bit** (standard, by far most common) | 128 possible addresses (16 reserved for special purposes, ~112 usable) | Single address byte: 7 address bits + 1 R/W bit |
| **10-bit** (rare, for large device counts) | 1024 possible addresses | A special reserved first-byte pattern ( `11110XX` + 2 MSBs of the 10-bit address + R/W), followed by a second byte carrying the remaining 8 address bits — takes two bytes total to address a device |

10-bit addressing exists for buses with many devices where 7-bit's ~112 usable addresses aren't enough, but it's rarely seen in practice — most real designs either use 7-bit addressing directly or use an I2C multiplexer/expander chip to work around address

space or address-collision limits instead.

## 3.3 Reading Multiple Bytes — Burst Read

After the first data byte, if the master keeps ACKing (instead of NACKing), the slave will typically keep auto-incrementing its internal register pointer and keep sending the next byte — this "burst read" pattern is extremely common for pulling out multi-byte sensor readings (e.g., a 3-axis accelerometer's 6 bytes of X/Y/Z data) in a single transaction: START, address+R, ACK, byte1, ACK, byte2, ACK, ... byteN, **NACK** (tells the slave "that was the last one, stop"), STOP.

> The final NACK (instead of ACK) on the last byte the master wants is deliberate and important — it's the master's way of telling the slave "don't send any more," since in read mode the slave is the one driving SDA and has no other way to know when the master is done.

# Part 4 — Clock Stretching, Multi-Master & Arbitration

## 4.1 Clock Stretching

Because SCL is open-drain (not exclusively master-driven the way SPI's SCLK is), a **slave is allowed to hold SCL LOW** after the master releases it, if the slave needs more time before it's ready for the next clock pulse (e.g., it's busy processing the previous byte internally). The master must check that SCL has actually gone HIGH (not just release it and assume) before proceeding — this is a mandatory part of a compliant I2C master implementation, though some simple/bit-banged master implementations skip this check and can misbehave with slaves that stretch the clock.

> This is the I2C equivalent of CAN TP's "WAIT" Flow Control frame or a UDS negative-response-pending — a receiver-side mechanism to buy time without aborting the transaction, but implemented at the electrical layer instead of a protocol message.

## 4.2 Multi-Master Arbitration

Since I2C explicitly supports multiple masters, it needs a way to resolve two masters starting a transaction at the same time — conceptually similar to CAN's bitwise arbitration, but applied to a single shared SDA line instead of a differential ID field:

- Every transmitting master also monitors SDA while it drives it (exactly like a CAN transmitter monitoring the bus during arbitration).
- As long as both masters happen to be sending identical bits, neither notices anything unusual.
- The moment one master tries to send a HIGH (release SDA) while the other sends a LOW (actively pulls SDA down), the wired-AND nature of the open-drain bus means the line reads LOW. The master that wanted HIGH sees a mismatch between what it drove and what's actually on the bus, recognizes it has lost arbitration, and immediately stops driving, backing off to wait for the bus to go idle again.
- The master sending LOW (numerically, this tends to favor whichever master's address/data bit pattern has more leading zeros — directly analogous to CAN's "lower ID wins") continues completely unaware that a collision even happened.

> This means, just like CAN, I2C arbitration is **non-destructive** — the winning master's transaction proceeds completely intact, and the losing master simply retries later. No data is corrupted; the losing master just doesn't get the bus this time around.

# Part 5 — Speed Modes & Practical Engineering Notes

## 5.1 I2C Speed Modes

| Mode | Max clock speed | Notes |
|---|---|---|
| Standard Mode | 100 kHz | The original I2C spec speed; still widely supported as a safe default |
| Fast Mode (Fm) | 400 kHz | Very common in modern embedded designs |
| Fast Mode Plus (Fm+) | 1 MHz | Requires stronger pull-ups / lower bus capacitance |
| High Speed Mode (Hs) | 3.4 MHz | Requires a special protocol prefix and different electrical characteristics (current-source pull-ups); rarely used outside specialized applications |

## 5.2 Common Bring-Up Issues

| Symptom | Likely Cause |
|---|---|
| Bus is stuck LOW permanently, nothing works | A slave crashed mid-transaction while holding SDA or SCL low (e.g., reset during a byte), or missing/broken pull-up resistors — a common recovery technique is for the master to manually clock up to 9 extra SCL pulses to force a stuck slave to release SDA, then issue a STOP |
| Master gets NACK on the address byte every time | Wrong 7-bit address (very common — some datasheets list the 8-bit form including R/W already shifted in, causing an off-by-one-bit address error), device not powered, or device held in reset |
| Works with one device on the bus, fails with several | Bus capacitance too high for the chosen pull-up value/speed once multiple device pins add parasitic capacitance, or an address collision between two devices that share the same fixed address (common with multiple identical sensor breakout boards — often solved via an address-select pin or an I2C multiplexer) |
| Random data corruption only under load / multiple masters | A master implementation that isn't correctly monitoring SDA for arbitration loss, or not correctly waiting for slave clock-stretching before proceeding |
| Communication fails only at higher clock speeds | Pull-up resistor value too high for the bus capacitance at that speed (rise time exceeds what the mode requires) — needs a smaller pull-up value or shorter/lower-capacitance wiring |

## 5.3 SPI vs I2C vs CAN — Quick Comparison

| Property | I2C | SPI | CAN |
|---|---|---|---|
| Wires | 2 (SDA, SCL), shared by all devices | 4 (+1 CS per extra slave) | 2 (CAN_H, CAN_L), differential, shared |
| Duplex | Half | Full | Half (one transmitter at a time, but broadcast to all) |
| Addressing | In-band, 7/10-bit address on the bus | Physical (CS line) — no in-band address | In-band message ID (content-based, not device-based) |
| Multi-master | Yes, with bitwise arbitration | No (single master by design) | Yes, with bitwise arbitration |
| Error checking | Per-byte ACK/NACK | None built-in | CRC + multiple error types + error frames |
| Typical speed | 100 kHz - 3.4 MHz | Several MHz - 10s of MHz | Up to 1 Mbit/s (classic), several Mbit/s (FD) |

# Part 6 — Interview Questions & Answers

## 6.1 I2C Fundamentals

**Q1. What are the two I2C signal lines and why are they open-drain?**

SDA (Serial Data) and SCL (Serial Clock). Both are open-drain so that multiple devices can share the same two wires without any risk of two outputs directly driving opposite logic levels against each other — a device can only pull a line low, never actively drive it high, so there's never electrical contention, only a wired-AND effect.

**Q2. What is the role of the pull-up resistors on SDA and SCL?**

Since open-drain outputs can only pull a line low, something has to pull it back up to a logic-high level when no device is actively pulling it low — that's the pull-up resistor's job. Its value is a real design tradeoff: too high slows the rising edge (limiting achievable clock speed for a given bus capacitance), too low increases current draw and can exceed a device's rated sink current.

**Q3. Is I2C full-duplex or half-duplex?**

Half-duplex — a single shared SDA line carries data in only one direction at any given moment (though the direction can switch multiple times within one transaction, e.g., via a repeated START).

**Q4. What is the fundamental timing rule that governs all of I2C's framing?**

SDA may only change while SCL is LOW; while SCL is HIGH, SDA must be stable, because a receiver is sampling it. The only exceptions are START and STOP, which are specifically defined as SDA transitions during SCL HIGH — precisely so they can never be mistaken for a data bit.

## 6.2 START/STOP & Framing

**Q5. Define the START and STOP conditions.**

START: SDA transitions HIGH-to-LOW while SCL is HIGH. STOP: SDA transitions LOW-to-HIGH while SCL is HIGH. START marks the bus going from idle to busy; STOP marks it returning to idle.

**Q6. What is a Repeated START and why would you use one?**

A Repeated START is issuing a new START condition without first issuing a STOP. It's used to combine multiple operations (most commonly a register-address write followed immediately by a data read) into one atomic transaction that another master can't interrupt, since the bus never goes idle between the two operations — releasing the bus with a full STOP would allow another master to grab it in between.

**Q7. Why does every I2C byte need to be followed by an ACK/NACK bit?**

It's the only built-in confirmation mechanism I2C has — since I2C has no CRC, the per-byte ACK is how the master knows a slave actually exists at that address and is receiving data correctly (for writes), or how a slave/master signals whether to continue or stop sending more bytes (for reads).

**Q8. What does it mean if a master gets a NACK immediately after sending the address byte?**

No device on the bus recognized that address — most commonly a wrong address, the intended device not being powered/present, or the device being held in reset. Since nothing pulls SDA low during the ACK slot, the pull-up keeps it high, which the master reads as a NACK.

## 6.3 Addressing & Frame Structure

**Q9. Walk through the byte-level structure of a basic I2C write transaction.**

START → 7-bit slave address + R/W=0 (write) as one byte → ACK from slave → data byte → ACK from slave → (repeat data+ACK for more bytes if needed) → STOP.

**Q10. What's the difference between 7-bit and 10-bit addressing?**

7-bit addressing (the standard, near-universal choice) fits the address and R/W bit into a single byte, giving ~112 usable addresses. 10-bit addressing uses a reserved first-byte pattern plus a second address byte to reach 1024 possible addresses, but is rarely used in practice — most designs work around address space limits with an I2C multiplexer instead.

**Q11. How does a master read multiple consecutive bytes (a "burst read") from a slave?**

After each received data byte, the master sends ACK to request the next byte (the slave auto-increments its internal register pointer and keeps sending), and sends NACK on the final byte it wants to signal the slave to stop — then issues STOP.

**Q12. Why is the R/W bit combined into the same byte as the address rather than sent separately?**

It keeps the addressing phase to exactly one byte (7 address bits + 1 R/W bit = 8 bits), minimizing per-transaction overhead — and it's also why the address+R/W byte is sometimes confusingly written as an "8-bit address" in some datasheets, which is really the 7-bit address pre-shifted left by one with R/W in bit 0 — a common source of off-by-one-bit addressing bugs when porting driver code between datasheets that use different conventions.

## 6.4 Clock Stretching & Multi-Master

**Q13. What is clock stretching and which device performs it?**

A slave device holding SCL LOW after the master releases it, to buy itself more time before the next clock pulse (e.g., while it's busy processing the previous byte internally). The master must actively check that SCL has gone high before continuing, rather than assuming it has, to correctly support slaves that stretch the clock.

**Q14. How does I2C multi-master arbitration work, and how is it similar to CAN?**

Every transmitting master monitors SDA while driving it. If a master sends a recessive-equivalent HIGH (releases SDA) but reads back LOW (because another master is actively pulling it low), it recognizes it lost arbitration and backs off, letting the other master continue uninterrupted. This is conceptually identical to CAN's non-destructive bitwise arbitration, just applied to a single open-drain SDA line instead of a differential ID field — in both cases, the "dominant" (actively driven low / low bit) always wins, and the losing side detects the mismatch by reading back what it actually sent versus what's really on the bus.

**Q15. What happens to the data of the master that loses arbitration?**

Nothing is corrupted — the losing master simply stops transmitting immediately upon detecting the mismatch and waits for the bus to go idle again before retrying its transaction; this is what makes I2C (like CAN) arbitration non-destructive.

## 6.5 Practical / Troubleshooting

**Q16. The I2C bus appears permanently stuck LOW and no device responds. How would you recover it, and why does this happen?**

It typically happens when a slave crashes or resets mid-transaction while it happens to be holding SDA (or SCL) low. A standard recovery technique is for the master to manually clock up to 9 extra SCL pulses (enough to complete any partial byte the stuck slave might be mid-way through) while watching for SDA to release, then issue a STOP condition to force the bus back to a known idle state.

**Q17. Two identical sensor breakout boards on the same I2C bus both have the fixed address 0x68. How do you resolve this?**

Common fixes: use a device variant/pin strap that offers an alternate address (many sensor ICs expose an address-select pin for exactly this reason), or insert an I2C multiplexer/switch IC between the master and the two devices so each can be selected onto a separate sub-bus one at a time.

**Q18. Communication is reliable at 100 kHz but fails intermittently at 400 kHz on the same hardware. What's the likely cause?**

The pull-up resistor value is likely too large for the bus's total capacitance at the faster required rise time — Fast Mode's tighter timing budget leaves less margin for a slow RC rise, so a value that worked fine at Standard Mode's more relaxed timing can fail once the clock speeds up. The fix is typically a smaller pull-up value or reducing bus capacitance (shorter wires, fewer devices, better layout).

**Q19. Why might a driver written for one I2C device datasheet fail when ported to what looks like an equivalent device from another vendor?**

A very common cause is an address convention mismatch — one datasheet might list the "7-bit address" directly, while another lists the same physical address already left-shifted into 8-bit form (with R/W as bit 0), and copying the numeric value verbatim without accounting for this produces a silently wrong address that either NACKs immediately or (worse) hits a different device's address by coincidence.

**Q20. What's the practical benefit of I2C's per-byte ACK over SPI having no acknowledgment at all?**

It gives the master immediate, built-in confirmation that a device exists and is responding at the address it just addressed — useful for bus scanning/device presence detection and catching wiring/addressing mistakes immediately — whereas SPI's lack of any acknowledgment means a completely disconnected or misconfigured slave can fail totally silently, with the master having no way to know from the bus itself whether anything actually received its data.

— End of Guide —